BACKEND · AUTHENTICATION

Login with Google, built from first principles

What OAuth actually is, what Passport.js does for you, and how one click turns into a logged-in user — with the "already registered? log in : register" logic wired up in Express and MongoDB.

Node.js | Express | Passport.js | OAuth 2.0 / OIDC | MongoDB | JWT

---

01 Why OAuth exists

The problem first — then the protocol makes sense

Imagine a photo-printing app that wants your Google Photos. The naive way: you type your Google email and password into the printing app. Now that app can read your Gmail, delete your Drive, change your password — everything. And you can't take access back without changing your password everywhere.

OAuth fixes this. Instead of sharing your password, Google hands the app a **limited, revocable key** — "this app may read your photos, nothing else, until you revoke it." Your password only ever goes to Google.

> KEY CONCEPT

OAuth 2.0 is a **delegated authorization** framework: it lets an app act on a user's behalf with limited permissions, without ever seeing the user's password. Think of a hotel key card — it opens your room for three nights, not the whole hotel forever.

Authentication vs Authorization

What	Authentication (AuthN)	Authorization (AuthZ)
Question	Who are you?	What are you allowed to do?
Example	Showing your ID at the gate	Your ticket says Gate B, Seat 14
Protocol	OpenID Connect (OIDC) ✓	OAuth 2.0 ✓
Token	`id_token` (a JWT about the user)	`access_token` (key to call APIs)

💡 **"Login with Google" is technically OpenID Connect** — a thin identity layer built on top of OAuth 2.0. When you request the `openid` / `profile` / `email` scopes, Google also tells you *who* the user is. Most people (and most interviewers) just say "OAuth", but knowing this distinction is an instant plus point.

OAuth 2.0, on its own, is primarily a framework for...

A — Authentication (proving who the user is)

B — Authorization (delegating limited access)

02 OAuth vocabulary

Every word you'll see in docs, code and interviews

The four roles

ROLE	IN OUR APP	WHAT IT DOES
Resource Owner	The user	Owns the Google account and decides whether to grant access.
Client	Our Express server	The app asking for access. Holds the `client_id` and `client_secret`.
Authorization Server	accounts.google.com	Shows the login + consent screen, issues the code and tokens.
Resource Server	Google APIs (userinfo)	Accepts the access token and returns protected data (name, email, photo).

The terms

TERM	EXAMPLE	WHAT IT MEANS
client_id	1234-abc.apps.googleusercontent.com	Public identifier of our app. Safe to appear in URLs.
client_secret	GOCSPX-…	Our app's password with Google. Lives only on the server, never in the browser or Git.
redirect_uri	/auth/google/callback	Where Google sends the user back. Must exactly match a URI registered in Google Cloud.
scope	profile email	What we're asking permission for. Ask for the minimum you need.
state	random string	CSRF protection. We send it, Google echoes it back, we check it matches.
authorization code	?code=4/0Ab…	Short-lived, single-use ticket. Worthless without the client secret.
access_token	ya29.a0…	Key to call Google APIs on the user's behalf. Expires in ~1 hour.
refresh_token	1//0g…	Gets new access tokens without asking the user again. Only issued for offline access.

TERM	EXAMPLE	WHAT IT MEANS
`id_token`	`eyJhbGci…`	OIDC JWT that describes the user (`sub`, email, name).

Grant types (flows)

Flow	Used by	Status
Authorization Code	Server-side apps that can keep a secret (our Express app)	What we're building ✓
Auth Code + PKCE	SPAs and mobile apps (can't hide a secret)	Recommended for public clients ✓
Client Credentials	Machine-to-machine, no user involved	Valid, different use case
Implicit	Old SPAs — token returned directly in the URL	Deprecated, don't use

Which of these values must never reach the browser?

A — client_id

B — client_secret

03 What is Passport.js

Authentication middleware that hides the OAuth plumbing

You *could* implement OAuth by hand: build the Google URL, generate and verify `state`, POST the code to Google's token endpoint, call the userinfo API, handle every error. Passport.js does all of that for you and gives you one place to write *your* logic.

> KEY CONCEPT

Passport.js is authentication middleware for Express. Each login method is a plug-in called a **strategy** — `passport-local` for email/password, `passport-google-oauth20` for Google, and 500+ others. Passport does **not** store users. It hands you the verified Google profile, and **you** decide what to do with your database.

The four pieces you'll touch

- `passport.use(new GoogleStrategy(options, verify))`
  Register the strategy once with your client ID, secret and callback URL.
- `passport.authenticate('google')`
  Route middleware. Without a `?code` it redirects to Google; with a `?code` it exchanges it and fetches the profile.
- **The verify callback —** `(accessToken, refreshToken, profile, done)`
  Your code. This is where "registered? log in : register" lives. Call `done(null, user)` when finished.
- `req.user`
  Whatever you passed to `done()` is attached here for the next handler.

By hand vs with Passport

Job	By hand	With Passport
Build consent URL	Manually assemble query params	Automatic ✓
Exchange code → tokens	POST to oauth2.googleapis.com/token	Automatic ✓
Fetch profile	GET userinfo with Bearer token	Automatic, normalized ✓
Find / create user	Your code	Still your code (verify callback)
Issue session / JWT	Your code	Passport sessions, or your JWT

Does Passport.js save your users to a database?

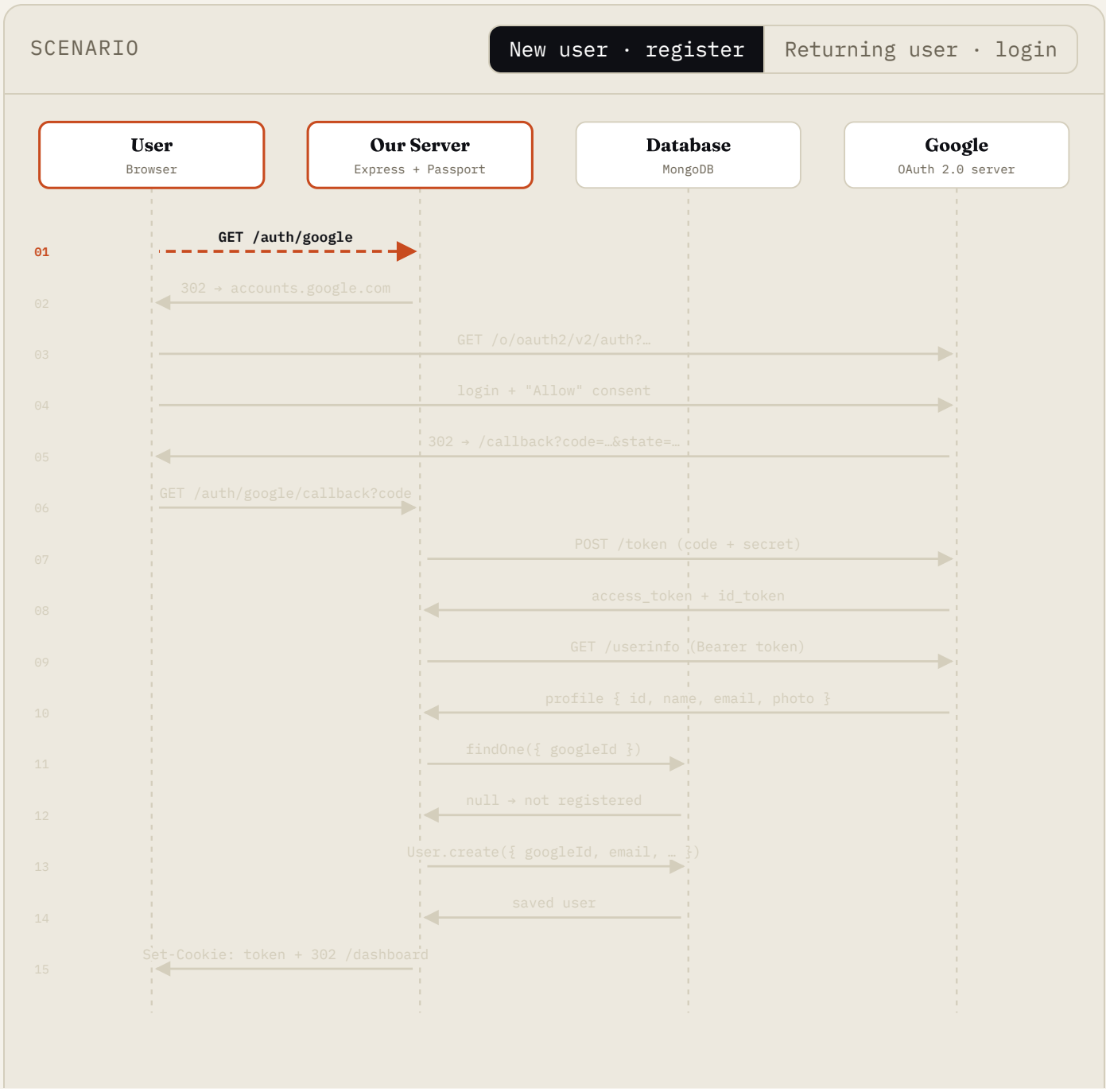
A — Yes, Passport keeps its own user store

B — No, you save them yourself inside the verify callback

04 The full flow

User → our server → database → Google, one step at a time

Step through the whole journey. Switch between a **new user** (gets registered) and a **returning user** (just logged in) to see exactly where the paths split. You can also click any arrow, or use the ← → keys.



STEP 1 FRONT-CHANNEL · VIA BROWSER

User clicks "Continue with Google"

A plain link (not axios) navigates the browser to our server. One button for both sign up and sign in — the user doesn't need to know which one it is.

```
<a href="http://localhost:3000/auth/google">Continue with Google</a>
```

← Back 01 / 15 Forward →

KEY CONCEPT

Front-channel = messages that travel through the user's browser via redirects (visible in the URL bar, history, logs). **Back-channel** = our server talking to Google directly. That's why Google only puts a short-lived **code** in the front-channel and hands out the real tokens on the back-channel, where the `client_secret` proves it's really us.

Who exchanges the authorization code for an access token?

A — The browser, right after the redirect

B — Our server, directly with Google

05 Login or register — the core logic

One endpoint, two outcomes, decided inside the verify callback

With Google login there's no separate "Sign up" and "Sign in" button. The user clicks one button, and our server decides: **seen this Google account before? Log in. Never seen it? Register, then log in.** This pattern is called *find-or-create*.

- **Google profile arrives**
  `profile.id`, `profile.displayName`, `profile.emails[0].value`, `profile.photos[0].value`

`User.findOne({ googleId: profile.id })`

Look up by Google's stable ID — not by name, and not by email alone.

`FOUND → REGISTERED`

Skip creation. This is a login.

`NULL → NOT REGISTERED`

`User.create({ googleId, email, name, avatar })`

`done(null, user)`

Both paths end the same way: we have a user document. Issue a session or JWT and redirect.

VERIFY CALLBACK — FIND-OR-CREATE COPY

```
async (accessToken, refreshToken, profile, done) => {
  try {
    // 1. Already registered? → log in
    let user = await User.findOne({ googleId: profile.id });
    if (user) return done(null, user);

    // 2. Not registered → register
    user = await User.create({
      googleId: profile.id,
      email: profile.emails[0].value,
      name: profile.displayName,
      avatar: profile.photos?.[0]?.value,
    });
    return done(null, user);
  } catch (err) {
    return done(err);
  }
}
```

Level up: account linking

What if Riya signed up last month with email + password, and today clicks "Continue with Google" with the same Gmail? The code above would try to create a second user and crash on the unique email index. The fix: if there's no `googleId` match, check for a **verified** email match and attach the Google ID to that existing account.

FIND → LINK → CREATE COPY

```
let user = await User.findOne({ googleId: profile.id });

if (!user) {
  const email = profile.emails?.[0]?.value;
  const emailVerified = profile._json?.email_verified;

  // Same verified email registered earlier with a password? Link it.
  if (email && emailVerified) user = await User.findOne({ email });

  if (user) {
    user.googleId = profile.id;
    await user.save();
  } else {
    user = await User.create({ googleId: profile.id, email, name: profile.displayName
  }
}
return done(null, user);
```

⚠️ **Never link accounts on an unverified email.** If you trust any email a provider sends, an attacker could create an account elsewhere with the victim's email and take over their account in your app.

💡 **Race condition:** two quick double-clicks can both see `null` and both try to create. The `unique` index on `googleId` makes the second insert fail instead of creating a duplicate. For a fully atomic version, use `findOneAndUpdate({ googleId }, { $setOnInsert: {...} }, { upsert: true, new: true })`.

What should you use to decide whether a Google user is already registered?

A — profile.id, stored as googleId

B — profile.displayName

06 Google Cloud Console setup

Getting a client ID and secret for our app

STEP 1 — CREATE A PROJECT

Open `console.cloud.google.com`, create a new project (e.g. *auth-demo*) and select it.

STEP 2 — CONFIGURE THE CONSENT SCREEN

Go to **APIs & Services** → **OAuth consent screen**. Choose *External*, add the app name, support email and the scopes `email`, `profile`, `openid`. While the app is in *Testing* mode, add your own Gmail under **Test users** — only those accounts can log in.

STEP 3 — CREATE CREDENTIALS

Credentials → **Create credentials** → **OAuth client ID** → **Web application.** Under *Authorized redirect URIs* add:

AUTHORIZED REDIRECT URIS — COPY

```
http://localhost:3000/auth/google/callback
https://api.yourdomain.com/auth/google/callback
```

VERIFY

Google shows a **Client ID** and **Client secret**. Copy both into `.env`.

.ENV — COPY

```
PORT=3000
MONGO_URI=mongodb://127.0.0.1:27017/auth-demo
GOOGLE_CLIENT_ID=xxxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=GOCSPX-xxxx
GOOGLE_CALLBACK_URL=http://localhost:3000/auth/google/callback
JWT_SECRET=replace-with-a-long-random-string
CLIENT_URL=http://localhost:5173
```

⚠️ `redirect_uri_mismatch` is the #1 error students hit. The callback URL in your code must match the registered one **character for character** — protocol (`http` vs `https`), port, path and trailing slash all count.

07 Project setup

Packages and folder structure

TERMINAL COPY

```
npm install express passport passport-google-oauth20 mongoose jsonwebtoken cookie-pars
# only if you choose server-side sessions instead of JWT
npm install express-session connect-mongo
```

PACKAGE	WHY WE NEED IT
passport	The core middleware engine.
passport-google-oauth20	The Google strategy (OAuth 2.0 + profile fetch).
jsonwebtoken	Sign our own JWT after Google confirms the user.
cookie-parser	Read that JWT back from the cookie on later requests.
cors	Let the React app (different port) send cookies to the API.

FOLDER STRUCTURE COPY

```
src/
├── config/passport.js          # strategy + verify callback
├── models/user.model.js        # User schema
├── routes/auth.routes.js       # /auth/google, /callback, /me, /logout
├── middlewares/auth.middleware.js
└── app.js
```

SRC/APP.JS COPY

```
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const cookieParser = require('cookie-parser');
const passport = require('./config/passport');
const authRoutes = require('./routes/auth.routes');
```

```
const app = express();
app.use(cors({ origin: process.env.CLIENT_URL, credentials: true }));
app.use(express.json());
app.use(cookieParser());
app.use(passport.initialize());

app.use('/auth', authRoutes);

mongoose.connect(process.env.MONGO_URI)
  .then(() => app.listen(process.env.PORT, () => console.log('API on', process.env.POR
```

08 User model & Google strategy

Where the find-or-create logic plugs in

SRC/MODELS/USER.MODEL.JS COPY

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  googleId: { type: String, unique: true, sparse: true }, // sparse: password users ha
  email:    { type: String, required: true, unique: true, lowercase: true },
  name:     String,
  avatar:   String,
  password: { type: String, select: false },              // only for email/password u
}, { timestamps: true });

module.exports = mongoose.model('User', userSchema);
```

💡 **Why `sparse: true`?** A normal unique index treats a missing field as `null`, so the second password-only user would clash with the first. Sparse indexes skip documents that don't have the field.

SRC/CONFIG/PASSPORT.JS COPY

```
const passport = require('passport');
const { Strategy: GoogleStrategy } = require('passport-google-oauth20');
const User = require('../models/user.model');

passport.use(new GoogleStrategy(
  {
    clientID: process.env.GOOGLE_CLIENT_ID,
    clientSecret: process.env.GOOGLE_CLIENT_SECRET,
    callbackURL: process.env.GOOGLE_CALLBACK_URL,
  },
  // Runs AFTER Passport exchanged the code and fetched the profile
  async (accessToken, refreshToken, profile, done) => {
    try {
      let user = await User.findOne({ googleId: profile.id });
      if (user) return done(null, user);                    // registered → login

      user = await User.create({                            // new → register
        googleId: profile.id,
        email: profile.emails[0].value,
        name: profile.displayName,
        avatar: profile.photos?.[0]?.value,
      });
      return done(null, user);
    } catch (err) {
      return done(err);
    }
  }
));

module.exports = passport;
```

Strategy options

OPTION	VALUE USED	WHAT IT DOES
clientID	env	Identifies our app to Google.
clientSecret	env	Used only in the back-channel code exchange.
callbackURL	/auth/google/callback	Sent as `redirect_uri`. Must match the console.

OPTION	VALUE USED	WHAT IT DOES
proxy	true	Trust `X-Forwarded-Proto` behind a load balancer so the callback becomes `https`.
state	true	Generates and verifies the CSRF `state` (needs a session store).

09 Auth routes

Two routes start and finish the dance, two more use the result

SRC/ROUTES/AUTH.ROUTES.JS COPY

```
const router = require('express').Router();
const jwt = require('jsonwebtoken');
const passport = require('../config/passport');
const { requireAuth } = require('../middlewares/auth.middleware');

// Steps 1–2: start the flow → Passport redirects to Google
router.get('/google',
  passport.authenticate('google', {
    scope: ['profile', 'email'],
    prompt: 'select_account',
    session: false,
  })
);

// Steps 6–15: Google sends the user back with ?code=
router.get('/google/callback',
  passport.authenticate('google', {
    session: false,
    failureRedirect: `${process.env.CLIENT_URL}/login?error=google`,
  }),
  (req, res) => {
    // req.user = whatever the verify callback passed to done(null, user)
    const token = jwt.sign({ id: req.user._id }, process.env.JWT_SECRET, { expiresIn:

    res.cookie('token', token, {
      httpOnly: true,
```

```
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'lax',
      maxAge: 7 * 24 * 60 * 60 * 1000,
    });
    res.redirect(`${process.env.CLIENT_URL}/dashboard`);
  }
);

router.get('/me', requireAuth, (req, res) => res.json({ user: req.user }));

router.post('/logout', (req, res) => {
  res.clearCookie('token');
  res.json({ message: 'Logged out' });
});

module.exports = router;
```

`passport.authenticate()` options

OPTION	VALUE USED	WHAT IT DOES
`scope`	`['profile','email']`	Permissions requested. Shown on the consent screen.
`prompt`	`'select_account'`	Always show the account picker — handy when users have multiple Gmails.
`session`	`false`	Don't create a Passport login session — we issue our own JWT.
`failureRedirect`	`CLIENT_URL/login`	Where to go if the user clicks Cancel or verification fails.

💡 **Same middleware, two jobs.** On `/auth/google` there's no `?code`, so Passport redirects to Google. On the callback there *is* a `?code`, so Passport exchanges it, fetches the profile and runs your verify callback.

Both routes call `passport.authenticate('google')`. How does Passport know which job to do?

A — It checks whether the request has a ?code= query param

B — The callback route registers a different strategy

10 Session vs JWT

What happens after Google says "yes, this is Riya"

Google's job ends once we know who the user is. Keeping them logged in on *our* app is our job, and there are two common ways.

What	Passport sessions	JWT in httpOnly cookie
Where state lives	Server (session store, e.g. MongoDB/Redis)	Inside the signed token
Cookie holds	A random session ID	The JWT itself
Needs serialize/deserialize	Yes	No ✓
Instant logout / revoke	Easy — delete the session ✓	Hard until it expires (needs a denylist)
Scaling across pods	Shared store required	Stateless ✓
Fits best	Server-rendered apps	MERN with a separate React app ✓

If you choose sessions

SESSION-BASED SETUP COPY

```
const session = require('express-session');
const MongoStore = require('connect-mongo');
```

```
app.use(session({
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  store: MongoStore.create({ mongoUrl: process.env.MONGO_URI }),
  cookie: { httpOnly: true, secure: process.env.NODE_ENV === 'production', sameSite: '
}));
app.use(passport.initialize());
app.use(passport.session());

// After login: what to store in the session → just the id
passport.serializeUser((user, done) => done(null, user.id));

// On every request: id from session → full user on req.user
passport.deserializeUser(async (id, done) => {
  try { done(null, await User.findById(id)); }
  catch (err) { done(err); }
});
```

KEY CONCEPT

serializeUser runs once at login and decides what goes *into* the session (keep it tiny — the id). **deserializeUser** runs on every later request and turns that id *back* into a full user on `req.user`.

You use `session: false` and issue your own JWT. Which Passport functions can you skip?

A — serializeUser and deserializeUser

B — The verify callback

11 Frontend & protected routes

Starting the login from React and guarding APIs

REACT — LOGIN BUTTON COPY

```
// ✅ Full-page navigation — the browser follows every redirect
<a href="http://localhost:3000/auth/google">Continue with Google</a>

// ✅ Same thing from a click handler
window.location.href = 'http://localhost:3000/auth/google';

// ❌ Will NOT work
axios.get('http://localhost:3000/auth/google');
```

⚠️ **Never start OAuth with axios/fetch.** The response is a 302 to Google's login *page*. An XHR can't render that page for the user, and the cross-origin redirect fails CORS. OAuth needs a real top-level browser navigation.

REACT — WHO'S LOGGED IN? COPY

```
// On app load / dashboard mount
const { data } = await axios.get('http://localhost:3000/auth/me', {
  withCredentials: true, // send the httpOnly cookie
});
setUser(data.user);
```

SRC/MIDDLEWARES/AUTH.MIDDLEWARE.JS COPY

```
const jwt = require('jsonwebtoken');
const User = require('../models/user.model');

async function requireAuth(req, res, next) {
  const token = req.cookies.token;
  if (!token) return res.status(401).json({ message: 'Not logged in' });

  try {
    const { id } = jwt.verify(token, process.env.JWT_SECRET);
    req.user = await User.findById(id);
    if (!req.user) return res.status(401).json({ message: 'User not found' });
    next();
  } catch {
    res.status(401).json({ message: 'Invalid or expired token' });
  }
```

```
}

// Session version is one line: if (req.isAuthenticated()) return next();
module.exports = { requireAuth };
```

How should the React app start Google login?

A — axios.get('/auth/google') and read the response

B — An <a href> link or window.location to /auth/google

12 Security & production gotchas

The things that work on localhost and break in prod

RISK	FIX	WHY
Login CSRF	state: true	Stops an attacker from completing the flow with *their* code in your browser, silently logging you into their account.
Leaked secret	.env + .gitignore	With the secret, anyone can exchange codes as your app. Rotate it immediately if pushed.
Token theft (XSS)	httpOnly cookie	JavaScript can't read it, unlike `localStorage`.
Cookie over HTTP	secure: true	Cookie only travels over HTTPS in production.
Cross-site requests	sameSite: 'lax'	Sent on the top-level redirect back from Google, blocked on most cross-site requests.
Over-asking	minimal scopes	Only `profile email` for login. Scary consent screens lose users.
Stored Google tokens	discard or encrypt	For login only, you don't need them. If you call Google APIs later, encrypt the refresh token at rest.

💡 **About `state` with JWT:** Passport stores the `state` value in `req.session`. So if you enable `state: true` while using `session: false`, add `express-session` just for the OAuth handshake — or use a custom state store.

Behind a load balancer (EKS, Render, Nginx)

Your Node app receives plain HTTP from the load balancer, even though the user is on HTTPS. Passport then builds `http://…/callback`, Google sees a URI you never registered, and you get `redirect_uri_mismatch` — only in production.

PRODUCTION FIX COPY

```
app.set('trust proxy', 1); // believe X-Forwarded-Proto from the LB / ingress

new GoogleStrategy({
  clientID: process.env.GOOGLE_CLIENT_ID,
  clientSecret: process.env.GOOGLE_CLIENT_SECRET,
  callbackURL: '/auth/google/callback', // relative…
  proxy: true,                          // …resolved using the forwarded https
}, verify);
```

⚠️ **Frontend and API on different sites?** (e.g. `app.vercel.app` + `api.render.com`) Browsers treat that cookie as third-party and increasingly block it. Put both on the same parent domain — `app.yourdomain.com` and `api.yourdomain.com` — and set `domain: '.yourdomain.com'` on the cookie.

The `state` parameter mainly protects against…

A — CSRF / login forgery on the callback

B — XSS stealing the access token

13 Interview questions

Tap a question, answer it out loud first, then check

FUNDAMENTALS

- Q1 What is OAuth 2.0? Is it authentication or authorization? — Basic
- Q2 Walk me through the Authorization Code flow end to end. — Basic
- Q3 Why does Google send a code instead of the access token directly? — Medium
- Q4 Access token vs refresh token vs id_token? — Basic
- Q5 What is the redirect_uri and why must it be pre-registered exactly? — Basic
- Q6 What is the state parameter for? — Medium
- Q7 What is PKCE and when do you need it? — Advanced
- Q8 What are scopes? Which ones do you need for login? — Basic

PASSPORT.JS

- Q9 What is Passport.js and what is a strategy? — Basic
- Q10 What does the verify callback do, and what is `done` ? — Medium
- Q11 Explain serializeUser and deserializeUser. — Medium

Q12 How do you use Passport without sessions? Medium

SCENARIOS & DESIGN

Q13 How do you decide whether to log in or register a Google user? Basic

Q14 Why look up by googleId and not by email? Medium

Q15 A user signed up with email/password and now clicks "Continue with Google" with the same email. What happens? Advanced

Q16 Two requests try to register the same new user at once. How do you avoid duplicates? Advanced

Q17 Where do you store the JWT after Google login, and why? Medium

Q18 Why can't the React app call `/auth/google` with axios? Medium

Q19 It works on localhost but gives redirect_uri_mismatch in production. Why? Advanced

Q20 Do you need to store Google's access token in your database? Medium

Q21 When a Google user clicks Logout in your app, are they logged out of Google? Basic

Q22 Your JWT is valid for 7 days. How do you force-logout a user right now? Advanced

14 Revision sheet

The whole topic in one screen, the night before the interview

```
Click "Continue with Google" → GET /auth/google
    ↓
302 to Google with client_id · redirect_uri · scope · state
    ↓
User logs in on Google (password never touches our server)
    ↓
302 back to /auth/google/callback?code=…&state=…
    ↓
Server ⇔ Google code + secret → tokens → profile (Passport does this)
    ↓
findOne({ googleId }) → found: login / null: create
    ↓
Set httpOnly cookie → redirect /dashboard ✅
```

Remember	One-liner
OAuth vs OIDC	OAuth = access, OIDC = identity on top
Why a code	Front-channel is leaky; tokens only on the back-channel with the secret
Passport's job	The OAuth plumbing. Your job: find-or-create + session/JWT
Lookup key	`googleId`, unique index; email only for verified linking
Start login	Top-level navigation, never axios
Prod bug #1	redirect_uri_mismatch → trust proxy + `proxy: true`

A returning user logs in with Google. Which database call happens?

A — findOne, then always create

B — Only findOne — the user is found, so no create

Which flow should a server-side Express app with a client secret use?

A — Implicit flow

B — Authorization Code flow